

SOFTWARE REQUIREMENTS SPECIFICATION

NeoBank Platform — Sprint 3 — Loan Management

Project	NeoBank Platform
Document	Software Requirements Specification (SRS)
Sprint	Sprint 3 — Loan Management
Duration	10 Working Days (Days 21–30 of 60-Day Lifecycle)
Program	PMIS (Pradhan Mantri Internship Scheme) — Infosys Limited
Location	Bhubaneswar Development Centre — Lab 4 & Lab 5
Version	1.0 — Draft, Internal Review
Date	May 2026
Status	Active — Sprint In Progress
Previous Sprint	Sprint 2 — Budgeting, Bills & Rewards (Days 11–20)
Next Milestone	Milestone 4 — Insights, Admin Controls & Stabilization

© Infosys Limited · Confidential · Not for External Distribution

1. INTRODUCTION

The NeoBank Platform is an enterprise-grade digital banking solution developed under the PMIS (Pradhan Mantri Internship Scheme) Internship Program at the Infosys Bhubaneswar Development Centre, Lab 4 and Lab 5. This document is the authoritative Software Requirements Specification (SRS) for Sprint 3, covering Days 21 through 30 of the 60-day development lifecycle.

With the core banking ledger established in Sprint 1 and personal finance tooling delivered in Sprint 2, this sprint shifts focus to credit facilities. Sprint 3 delivers an end-to-end loan lifecycle system encompassing loan product configuration, user loan applications, administrative approval workflows, EMI calculation, and repayment schedule tracking — together forming a comprehensive credit management layer on the NeoBank Platform.

1.1 Purpose

This SRS defines the complete set of functional requirements, system behaviour, business rules, integration strategy, quality assurance plan, and sprint-level execution roadmap for NeoBank Sprint 3. It serves as the binding technical reference for all interns, mentors, and stakeholders in Lab 4 and Lab 5.

1.2 Scope of Sprint 3

Sprint 3 is scoped exclusively to the Loan Management module. Deliverables are limited to:

- FR-5: Loan Management — loan product configuration, user loan applications, admin approval workflow, EMI calculation, and repayment schedule generation and tracking
- Database schema extension: provisioning LoanProduct, LoanApplication, LoanAccount, and LoanRepayment tables via DDL scripts
- Frontend-backend integration: Angular components wired to Spring Boot REST APIs with JWT-secured and role-enforced requests
- Unit and end-to-end testing of all sprint deliverables, with strict mathematical validation of EMI logic
- Swagger/OpenAPI documentation updated for all new /api/loans/... endpoints

1.3 Project Context

Internship Program	PMIS (Pradhan Mantri Internship Scheme)
Organisation	Infosys Limited
Development Centre	Bhubaneswar DC
Labs	Lab 4 & Lab 5
Project	NeoBank Platform
Sprint Duration	10 Working Days (Days 21–30)
Total Cycle	60 Working Days
Previous Sprint	Sprint 2 — Budgeting, Bills & Rewards (Days 11–20)
Next Sprint	Milestone 4 — Insights, Admin Controls & Stabilization

1.4 Definitions & Abbreviations

TERM	DEFINITION
SRS	Software Requirements Specification — this document
FR	Functional Requirement — a categorised set of deliverable features
JWT	JSON Web Token — signed stateless token used for API authentication
RBAC	Role-Based Access Control — access restrictions based on assigned roles
DDL	Data Definition Language — SQL scripts to create and extend database schema
API	Application Programming Interface — RESTful HTTP endpoints
ORM	Object-Relational Mapping — Spring Data JPA maps Java entities to MySQL
DTO	Data Transfer Object — decouples API payload structure from JPA entity
EMI	Equated Monthly Instalment — fixed monthly repayment amount calculated on loan approval
PMIS	Pradhan Mantri Internship Scheme — government internship program hosted by Infosys
DC	Development Centre — Infosys Bhubaneswar facility

2. FUNCTIONAL REQUIREMENTS OVERVIEW

Sprint 3 delivers one core Functional Requirement module that extends NeoBank into the credit domain. FR-5 depends on the JWT authentication infrastructure from Sprint 1 and must be fully functional and tested before sprint sign-off.

2.1 FR-5: Loan Management

This module handles the entire credit lifecycle, from product configuration through to repayment tracking. It introduces a dedicated admin workflow for loan approvals and implements complex EMI financial calculations with a full amortization schedule persisted in the database.

2.1.1 Loan Product Configuration

- ▶ Admin users configure available loan products via POST /api/loans/products
- ▶ Each loan product defines: product name, minimum and maximum loan amount, base interest rate (annual), and list of allowed tenures in months
- ▶ Only users with the ADMIN role may create or modify loan products — CUSTOMER access returns HTTP 403 Forbidden
- ▶ All users may retrieve available loan products via GET /api/loans/products
- ▶ Loan product data is used to validate subsequent user applications — amounts and tenures outside configured limits are rejected

2.1.2 Loan Application Engine

- ▶ Authenticated CUSTOMER users submit loan applications via POST /api/loans/apply
- ▶ The application payload includes: selected loan product ID, requested amount, and requested tenure in months
- ▶ Requested amount must fall strictly within the minimum and maximum limits of the selected loan product
- ▶ Requested tenure must exactly match one of the allowed tenures configured for the selected product
- ▶ Application ownership is resolved exclusively from the JWT payload — userId never accepted from the request body
- ▶ Successful submission returns HTTP 201 Created with application details and status PENDING
- ▶ A user may not submit a duplicate application for the same product while a PENDING application already exists

2.1.3 Administrative Approval Workflow

- ▶ Admin users review and action loan applications via PUT /api/loans/{loanApplicationId}/decision
- ▶ Decision payload includes: decision (APPROVED / REJECTED) and optional admin remarks
- ▶ Only users with the ADMIN role may execute this endpoint — CUSTOMER access returns HTTP 403 Forbidden
- ▶ Approved applications trigger automatic LoanAccount creation and EMI schedule generation
- ▶ Rejected applications update application status to REJECTED with the provided remarks — no further processing occurs
- ▶ An application already in APPROVED or REJECTED state cannot be actioned again — HTTP 409 Conflict returned

2.1.4 EMI Calculation & Amortization Schedule

- ▶ EMI is calculated using the standard reducing balance formula: $EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$, where P = principal, r = monthly interest rate, n = tenure in months
- ▶ Upon loan approval, a LoanAccount record is automatically created with the approved principal, interest rate, tenure, and computed EMI

- ▶ The complete amortization schedule is generated and persisted to the LoanRepayment table — one record per instalment
- ▶ Each repayment record includes: instalment number, due date, EMI amount, principal component, interest component, and payment status (PENDING)
- ▶ EMI calculations must remain mathematically consistent across the entire repayment schedule — rounding adjustments applied to the final instalment

2.1.5 Repayment Tracking

- ▶ Users and admins retrieve the repayment schedule via GET `/api/loans/{loanAccountId}/repayments`
- ▶ Each instalment record reflects current payment status: PENDING, PAID, or OVERDUE
- ▶ OVERDUE status is automatically applied when a repayment due date passes without payment
- ▶ Cross-user access to repayment schedules is prohibited — JWT `userId` validated against `LoanAccount` ownership; HTTP 403 returned on mismatch

3. BUSINESS & VALIDATION RULES

The following rules are hard constraints enforced at the backend API layer, independent of any frontend validation. Any API response that violates these rules is classified as a defect.

FR-5 — LOAN APPLICATION RULES

REF	CONSTRAINT
BR-01	The requested loan amount must fall strictly within the minimum and maximum limits of the selected loan product. Out-of-range amounts return HTTP 400 Bad Request with a descriptive error body.
BR-02	The requested tenure must exactly match one of the allowed tenures configured for the selected loan product. Unrecognised tenure values return HTTP 400 Bad Request.
BR-03	Loan application ownership must be resolved from the Spring SecurityContext JWT payload — the userId must never be accepted from the HTTP request body.
BR-04	A user may not submit a duplicate application for the same loan product while an existing PENDING application is still open. Such requests return HTTP 409 Conflict.
BR-05	All secured loan application endpoints require a valid, non-expired JWT Bearer token in the Authorization header. Missing or invalid tokens return HTTP 401 Unauthorized.
BR-06	Cross-user access to loan application data is strictly prohibited. Requests for another user's applications return HTTP 403 Forbidden.

FR-5 — APPROVAL & PROCESSING RULES

REF	CONSTRAINT
BR-01	Only users with the explicit ADMIN role may approve or reject loan applications. CUSTOMER role access to the decision endpoint returns HTTP 403 Forbidden.
BR-02	An application already in APPROVED or REJECTED state cannot be actioned again. Repeated decision requests return HTTP 409 Conflict.
BR-03	EMI calculations must remain mathematically consistent across the entire repayment schedule. Rounding discrepancies must be absorbed by the final instalment only.
BR-04	LoanAccount creation and amortization schedule generation must execute atomically within a single @Transactional boundary upon approval. Partial state is not permitted.
BR-05	All repayment records are immutable once generated. No PUT or DELETE endpoints shall exist for LoanRepayment resources.
BR-06	Loan product configuration (minimum/maximum amount, interest rate, allowed tenures) is admin-only. CUSTOMER role access to product write endpoints returns HTTP 403 Forbidden.

4. INFRASTRUCTURE & ARCHITECTURE

Sprint 3 extends the same modern, loosely-coupled, three-tier full-stack architecture established in Sprints 1 and 2. No architectural changes are introduced; the Loan Management module is added as an extension to the existing layer structure with strict role-based access controls enforced at the Spring Security layer.

4.1 Technology Stack

LAYER	TECHNOLOGY	DETAILS
Frontend	Angular	TypeScript SPA; Angular CLI; Angular Material data tables for repayment schedule grids; Reactive Forms; HTTP Interceptors; Route Guards
Backend	Spring Boot	Java REST API; Spring Security with <code>@PreAuthorize</code> for Admin-only endpoints; Spring Data JPA; Hibernate ORM; Maven; <code>@Transactional</code> for atomic loan account + schedule generation
Database	MySQL 8.0+	Relational RDBMS; InnoDB engine; DDL scripts extending Sprint 2 schema; FK constraints across LoanProduct, LoanApplication, LoanAccount, LoanRepayment
Security	JWT + BCrypt	Stateless token auth inherited from Sprint 1; Bearer scheme; ADMIN role enforced via <code>@PreAuthorize</code> on decision and product-write endpoints
Version Control	Git	Feature-branch workflow; Sprint 3 branches from develop; all work on feature/sprint3-* branches
API Docs	Swagger/OpenAPI	springdoc-openapi extended with FR-5 loan endpoints; interactive UI at /swagger-ui.html
IDE	IntelliJ + VSCode	IntelliJ for Spring Boot; VS Code with Angular Language Service for frontend

4.2 Database Schema Extensions

Four new highly-relational tables are provisioned on Day 21 via DDL scripts, extending the existing Sprint 2 schema. Foreign key constraints are enforced with the InnoDB engine.

4.2.1 loan_products

- ▶ `id` — BIGINT, PK, AUTO_INCREMENT
- ▶ `product_name` — VARCHAR(255), NOT NULL, UNIQUE
- ▶ `min_amount` — DECIMAL(15,2), NOT NULL, CHECK (`min_amount > 0`)
- ▶ `max_amount` — DECIMAL(15,2), NOT NULL, CHECK (`max_amount > min_amount`)
- ▶ `annual_interest_rate` — DECIMAL(5,2), NOT NULL, CHECK (`annual_interest_rate > 0`)
- ▶ `allowed_tenures` — VARCHAR(255), NOT NULL (comma-separated months, e.g., '12,24,36,60')
- ▶ `created_at` — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

4.2.2 loan_applications

- ▶ `id` — BIGINT, PK, AUTO_INCREMENT
- ▶ `user_id` — BIGINT, FK → `users(id)`, ON DELETE CASCADE
- ▶ `loan_product_id` — BIGINT, FK → `loan_products(id)`, ON DELETE RESTRICT
- ▶ `requested_amount` — DECIMAL(15,2), NOT NULL
- ▶ `requested_tenure_months` — INT, NOT NULL
- ▶ `status` — ENUM('PENDING', 'APPROVED', 'REJECTED'), DEFAULT 'PENDING'

- ▶ admin_remarks — VARCHAR(500)
- ▶ applied_at — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
- ▶ decided_at — TIMESTAMP, NULL

4.2.3 loan_accounts

- ▶ id — BIGINT, PK, AUTO_INCREMENT
- ▶ loan_application_id — BIGINT, FK → loan_applications(id), UNIQUE, ON DELETE RESTRICT
- ▶ user_id — BIGINT, FK → users(id), ON DELETE CASCADE
- ▶ principal_amount — DECIMAL(15,2), NOT NULL
- ▶ annual_interest_rate — DECIMAL(5,2), NOT NULL
- ▶ tenure_months — INT, NOT NULL
- ▶ emi_amount — DECIMAL(15,2), NOT NULL
- ▶ disbursed_at — TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

4.2.4 loan_repayments

- ▶ id — BIGINT, PK, AUTO_INCREMENT
- ▶ loan_account_id — BIGINT, FK → loan_accounts(id), ON DELETE CASCADE
- ▶ instalment_number — INT, NOT NULL
- ▶ due_date — DATE, NOT NULL
- ▶ emi_amount — DECIMAL(15,2), NOT NULL
- ▶ principal_component — DECIMAL(15,2), NOT NULL
- ▶ interest_component — DECIMAL(15,2), NOT NULL
- ▶ payment_status — ENUM('PENDING', 'PAID', 'OVERDUE'), DEFAULT 'PENDING'
- ▶ paid_at — TIMESTAMP, NULL

5. INSTALLATION & SETUP — DAY 21

Day 21 is dedicated to Sprint 3 environment extension and credit data modelling. Both sessions must be completed before Day 22 development begins. A fully extended schema with all four loan tables is a hard prerequisite for all subsequent sprint activity.

5.1 Session 1: Sprint 3 Branch & Frontend Dependencies

Git Branch Setup

- ▶ Create a new Git branch from develop: `git checkout -b feature/sprint3-loan-management`
- ▶ Push the branch to remote: `git push -u origin feature/sprint3-loan-management`
- ▶ Share branch access with all team members and the PMIS mentor

Angular Material & UI Library Setup

- ▶ Verify Angular Material is installed: `ng add @angular/material` (if not already present from prior sprints)
- ▶ Import `MatTableModule`, `MatPaginatorModule`, `MatSortModule` in `AppModule` — required for repayment schedule data grids
- ▶ Import `MatStepperModule` for multi-step loan application form
- ▶ Run `ng serve` to confirm the Angular application starts without errors after module additions

5.2 Session 2: Database Schema Extension

DDL Execution Order

- ▶ `07_create_loan_products_table.sql` — includes `UNIQUE` constraint on `product_name`
- ▶ `08_create_loan_applications_table.sql` — `FK` to `users` and `loan_products`; `ENUM` status
- ▶ `09_create_loan_accounts_table.sql` — `UNIQUE FK` to `loan_applications`; one account per approved application
- ▶ `10_create_loan_repayments_table.sql` — `FK` to `loan_accounts`; composite index on (`loan_account_id`, `instalment_number`)
- ▶ Verify: `SHOW TABLES`; confirms all 10 tables present
- ▶ Verify: `DESCRIBE loan_products`; `DESCRIBE loan_applications`; `DESCRIBE loan_accounts`; `DESCRIBE loan_repayments`; — validate all columns

Spring Boot Entity & DTO Setup

- ▶ Create `LoanProduct`, `LoanApplication`, `LoanAccount`, `LoanRepayment` JPA entities with `@Entity`, `@Table`, `@Id` annotations
- ▶ Create `LoanProductRepository`, `LoanApplicationRepository`, `LoanAccountRepository`, `LoanRepaymentRepository` extending `JpaRepository`
- ▶ Create corresponding DTOs: `LoanProductDTO`, `LoanApplicationRequestDTO`, `LoanApplicationResponseDTO`, `LoanDecisionDTO`, `LoanAccountDTO`, `RepaymentScheduleDTO`
- ▶ Create `EmiCalculatorUtil` class with static `calculateEMI(principal, annualRate, tenureMonths)` method
- ▶ Configure Spring Boot to validate schema against existing DDL with `hibernate.ddl-auto=validate`

6. SPRINT 3 EXECUTION PLAN

The 10-day sprint enables parallel backend and frontend development. An integration session on Day 28 bridges both layers. Each session is approximately 3–4 hours of focused work.

6.1 Sprint Schedule

DAY	PHASE	SESSION 1	SESSION 2
21	Env Setup	Create Sprint 3 Git branch; install Angular Material modules for data tables and stepper	Execute DDL scripts for all 4 loan tables; create JPA entities, repositories, and DTOs
22	Loan Products	POST /api/loans/products — admin-only loan product creation; GET /api/loans/products — public listing	Angular Admin UI: loan product configuration form with amount limits, interest rate, and tenure inputs
23	Application Engine	POST /api/loans/apply — user loan application with product and tenure validation	Angular Customer UI: multi-step application form with product selection, amount input, and submission
24	Approval Workflow	PUT /api/loans/{id}/decision — admin-only approval/rejection; strict RBAC enforcement	Angular Admin Dashboard: pending applications list, decision form with remarks, approve/reject actions
25	EMI & Accounts	LoanAccount auto-creation on approval; EMI CalculatorUtil with reducing balance formula	Amortization schedule generation; persist full LoanRepayment table with principal/interest split per instalment
26	Repayment API	GET /api/loans/{loanAccountId}/repayments — paginated schedule fetch with ownership check	Payment status reflection: PENDING, PAID, OVERDUE logic; due-date-based overdue detection
27	Repayment UI	Angular data grid: active loan accounts list with approved amount, EMI, and tenure	Detailed repayment schedule view: instalment number, due date, EMI, principal/interest split, status
28	Integration	Wire Angular loan application forms to Spring Boot endpoints; verify JWT user ID propagation	Connect Angular Admin decision components; verify EMI schedule generation triggers on approval state change
29	QA	JUnit tests for EMI calculator; security tests for Admin-only endpoints; application validation tests	End-to-end: apply as user → approve as admin → verify repayment schedule accuracy
30	Wrap-Up	Update Swagger/OpenAPI for all /api/loans/... endpoints; export openapi-sprint3.json	Final code review of EMI logic and Admin dashboards; QnA session; sprint sign-off; prep for final sprint

6.2 Day 22: Loan Product Configuration

Session 1 — Backend: Loan Product APIs

- ▶ Create LoanProduct JPA entity with @Entity, @Table; UNIQUE constraint on product_name
- ▶ LoanProductRepository extends JpaRepository<LoanProduct, Long>
- ▶ LoanProductService.create(): ADMIN role check via SecurityContextHolder → validate fields → save → return LoanProductDTO
- ▶ POST /api/loans/products — @PreAuthorize("hasRole('ADMIN')") → 201 Created or 400/403
- ▶ GET /api/loans/products — open to all authenticated users → return list of LoanProductDTO

Session 2 — Frontend: Admin Product Configuration UI

- ng generate component admin/loan-products
- Reactive FormGroup: productName, minAmount, maxAmount, annualInterestRate, allowedTenures (multi-select or comma-input)
- Validators: required, min(1) for amounts, maxAmount > minAmount cross-field validator
- On submit: LoanProductService.create() via HttpClient; display success or field errors
- Product listing table shows all configured products for admin reference

6.3 Day 23: Loan Application Engine

Session 1 — Backend: POST /api/loans/apply

- LoanApplicationService.apply(): extract userId from SecurityContextHolder → fetch LoanProduct → validate amount within [minAmount, maxAmount] → validate tenure in allowedTenures → check for duplicate PENDING application → save → return LoanApplicationResponseDTO
- LoanApplicationController @PostMapping("/api/loans/apply") → 201 Created or 400/409
- GET /api/loans/my-applications — returns all applications for the authenticated user

Session 2 — Frontend: Customer Application UI

- ng generate component loans/apply
- Step 1: product selection dropdown populated from GET /api/loans/products
- Step 2: dynamic amount input with min/max hints from selected product; tenure dropdown populated from allowedTenures
- Step 3: review and submit; display 201 confirmation or validation errors

6.4 Day 24: Administrative Approval Workflow

Session 1 — Backend: PUT /api/loans/{loanApplicationId}/decision

- @PreAuthorize("hasRole('ADMIN')") enforced — CUSTOMER access returns HTTP 403
- LoanDecisionService.decide(): fetch application → validate status is PENDING (HTTP 409 if already decided) → apply decision
- On APPROVED: update application status → trigger LoanAccountService.createAccount() + RepaymentScheduleService.generateSchedule() within @Transactional
- On REJECTED: update status to REJECTED with adminRemarks → no further processing
- GET /api/loans/admin/applications — @PreAuthorize("hasRole('ADMIN')") — all applications with status filter support

Session 2 — Frontend: Admin Decision Dashboard

- ng generate component admin/loan-decisions
- Paginated table of PENDING applications: applicant name, product, amount, tenure, applied date
- Decision modal: remarks text area, APPROVE and REJECT buttons
- On decision submit: PUT call → optimistic UI update → remove from PENDING list

6.5 Day 25: EMI Calculation & Loan Account Creation

Session 1 — Backend: EmiCalculatorUtil & LoanAccount

- EmiCalculatorUtil.calculateEMI(double principal, double annualRate, int tenureMonths):
- $r = \text{annualRate} / 12 / 100$ (monthly rate)
- $\text{EMI} = \text{principal} \times r \times (1+r)^n / ((1+r)^n - 1)$
- Return value rounded to 2 decimal places using HALF_UP rounding

- ▶ `LoanAccountService.createAccount()`: instantiate `LoanAccount` with calculated EMI → persist → return `LoanAccountDTO`

Session 2 — Backend: Amortization Schedule Generation

- ▶ `RepaymentScheduleService.generateSchedule(LoanAccount account)`:
- ▶ For each instalment i from 1 to `tenureMonths`:
- ▶ $\text{interestComponent} = \text{outstandingPrincipal} \times \text{monthlyRate}$
- ▶ $\text{principalComponent} = \text{EMI} - \text{interestComponent}$
- ▶ $\text{outstandingPrincipal} -= \text{principalComponent}$
- ▶ $\text{dueDate} = \text{disbursedAt} + i \text{ months}$
- ▶ Final instalment absorbs rounding residual to ensure $\text{sum of principalComponents} = \text{principal}$
- ▶ All `LoanRepayment` records persisted within same `@Transactional` as `LoanAccount` creation

6.6 Day 26: Repayment Tracking API

Session 1 — Backend: GET `/api/loans/{loanAccountId}/repayments`

- ▶ `LoanRepaymentService.getSchedule()`: validate JWT `userId` matches `LoanAccount.userId` → HTTP 403 if mismatch
- ▶ Return paginated list of `RepaymentScheduleDTO` sorted by `instalment_number` ascending
- ▶ Supports query params: `?status=PENDING|PAID|OVERDUE` for filtered views

Session 2 — Backend: Payment Status Logic

- ▶ Scheduled job (or on-fetch logic): compare `due_date` against current date → update `PENDING` → `OVERDUE` where $\text{due_date} < \text{today}$
- ▶ `PAID` status updated via `PATCH /api/loans/{loanAccountId}/repayments/{repaymentId}/pay` (admin or user action)
- ▶ Immutability enforced: no `PUT` or `DELETE` endpoints for repayment records

6.7 Day 27: Repayment Schedule UI

Session 1 — Active Loan Accounts View

- ▶ ng generate component `loans/my-loans`
- ▶ Angular Material table: loan account ID, product name, principal, EMI, tenure, disbursement date
- ▶ Row click navigates to detailed repayment schedule for the selected loan account

Session 2 — Repayment Schedule Detail View

- ▶ ng generate component `loans/repayment-schedule`
- ▶ Angular Material data grid columns: Instalment #, Due Date, EMI Amount, Principal, Interest, Status
- ▶ Status badges: `PENDING` → amber, `PAID` → green, `OVERDUE` → red via Angular class binding
- ▶ `MatPaginator` for large schedules (e.g., 60-month loans); `MatSort` on due date and status columns

7. API REFERENCE

All secured endpoints require a valid JWT Bearer token in the Authorization header. Endpoints inherited from Sprints 1 and 2 remain unchanged and are not repeated here.

FR-5 — LOAN PRODUCT ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/loans/products	POST	Day 22	ADMIN	Create a loan product with amount limits, interest rate, and tenures. Returns 201.
GET /api/loans/products	GET	Day 22	JWT	List all available loan products. Accessible to all authenticated users.
GET /api/loans/products/{id}	GET	Day 22	JWT	Fetch a specific loan product by ID. 404 if not found.

FR-5 — LOAN APPLICATION ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
POST /api/loans/apply	POST	Day 23	JWT	Submit a loan application. Validates amount and tenure against product config. Returns 201.
GET /api/loans/my-applications	GET	Day 23	JWT	List all loan applications for the authenticated user.
GET /api/loans/admin/applications	GET	Day 24	ADMIN	List all applications with optional status filter. Admin only.
PUT /api/loans/{loanApplicationId}/decision	PUT	Day 24	ADMIN	Approve or reject a PENDING application. Triggers account + schedule creation on approval.

FR-5 — LOAN ACCOUNT & REPAYMENT ENDPOINTS

ENDPOINT	METHOD	DAY	AUTH	DESCRIPTION
GET /api/loans/my-accounts	GET	Day 26	JWT	List all active loan accounts for the authenticated user.
GET /api/loans/{loanAccountId}/repayments	GET	Day 26	JWT	Paginated repayment schedule sorted by instalment number. 403 if not owner.
PATCH /api/loans/{loanAccountId}/repayments/{repaymentId}/pay	PATCH	Day 26	JWT	Mark a specific instalment as PAID. Validates ownership and current status.

8. INTEGRATION STRATEGY

The integration session on Day 28 bridges Angular and Spring Boot for all Sprint 3 modules. This session validates end-to-end data flow from the user interface through the API layer to MySQL and back, building on the JWT infrastructure verified in Sprints 1 and 2.

8.1 Day 28 — Sprint 3 Full Integration

Session 1 — Customer Loan Application Integration

- ▶ Wire Angular LoanService to POST /api/loans/apply and GET /api/loans/products endpoints
- ▶ Confirm LoanApplicationRequestDTO fields map correctly between Angular multi-step FormGroup and Spring Boot request body
- ▶ Verify JWT from sessionStorage is attached by JwtInterceptor on all loan API calls
- ▶ Test product validation end-to-end: select product → enter out-of-range amount → verify HTTP 400 displayed in UI with descriptive message
- ▶ Test tenure validation: select unlisted tenure value → verify HTTP 400 displayed in UI
- ▶ Verify my-applications list reflects submitted applications with correct PENDING status

Session 2 — Admin Decision & Schedule Integration

- ▶ Connect Angular Admin decision dashboard to GET /api/loans/admin/applications and PUT /api/loans/{id}/decision endpoints
- ▶ Test approval flow end-to-end: approve a PENDING application → verify LoanAccount created in DB → verify LoanRepayment records generated with correct instalment count
- ▶ Cross-verify EMI amount in Angular repayment grid against manual calculation: $EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$
- ▶ Verify rejection flow: reject an application → confirm status REJECTED in DB → verify no LoanAccount or LoanRepayment records created
- ▶ Test repeated decision attempt: action an already-APPROVED application → assert HTTP 409 Conflict displayed in Admin UI
- ▶ Test CUSTOMER role attempting approval endpoint → assert HTTP 403 displayed in UI
- ▶ Verify repayment schedule renders in Angular grid with correct number of rows matching tenure

9. DATA SEEDING

SQL seeding scripts populate the new Sprint 3 tables after schema creation, enabling integration and QA sessions to proceed with consistent, representative test data. Sprint 1 and Sprint 2 seed data is preserved and extended.

ENTITY	COUNT	SEED DETAILS
Loan Products	3+	Personal Loan (8% p.a., ₹50k–₹5L, tenures: 12/24/36); Home Loan (7% p.a., ₹10L–₹50L, tenures: 60/120/180); Auto Loan (9% p.a., ₹1L–₹10L, tenures: 24/36/48)
Loan Applications	4+	2 PENDING applications (one per customer user) for immediate Admin dashboard testing; 1 APPROVED application with linked LoanAccount; 1 REJECTED application with remarks
Loan Accounts	1+	One seeded LoanAccount linked to the APPROVED application; includes pre-calculated EMI amount for reference during QA
Loan Repayments	12+	Full amortization schedule seeded for the LoanAccount (minimum 12 instalments); mix of PENDING and at least 1 PAID status for testing status rendering
Test Credentials	N/A	admin@neobank.in / Admin@123 (ADMIN role) and customer1@neobank.in / Customer@123 (CUSTOMER role) from Sprint 1 seed retained

10. QUALITY ASSURANCE & TESTING

QA is integrated into Day 29. Testing covers the complete Loan Management module with a particular emphasis on mathematical precision of EMI calculations and strict enforcement of Admin-only access controls. All tests must pass before sprint sign-off on Day 30.

DAY 29 — FR-5 LOAN MANAGEMENT QA

Unit Tests

- ▶ EMI calculation — known values: assert `EmiCalculatorUtil.calculateEMI(100000, 10, 12)` matches expected output to 2 decimal places
- ▶ EMI schedule consistency: assert sum of all `principalComponents` in generated schedule equals original principal (within rounding tolerance)
- ▶ EMI schedule count: assert exactly `tenureMonths` `LoanRepayment` records generated per approved loan
- ▶ Loan application — amount below minimum: assert HTTP 400 returned before any DB write
- ▶ Loan application — amount above maximum: assert HTTP 400 returned before any DB write
- ▶ Loan application — invalid tenure: assert HTTP 400 returned
- ▶ Loan application — duplicate PENDING: assert HTTP 409 Conflict
- ▶ Approval endpoint — CUSTOMER role: assert HTTP 403 Forbidden
- ▶ Approval endpoint — already APPROVED application: assert HTTP 409 Conflict
- ▶ Repayment schedule — cross-user access: different JWT `userId` → assert HTTP 403 Forbidden
- ▶ Product creation — CUSTOMER role: assert HTTP 403 Forbidden

End-to-End Tests

- ▶ Full lifecycle: Login as CUSTOMER → browse loan products → apply for Personal Loan → Login as ADMIN → approve application → verify `LoanAccount` in DB → verify repayment schedule row count matches tenure
- ▶ Rejection flow: Login as ADMIN → reject a PENDING application with remarks → verify REJECTED status in Angular → confirm no `LoanAccount` or `LoanRepayment` records in DB
- ▶ EMI cross-check: compare Angular repayment schedule EMI against manual formula calculation for the seeded loan product
- ▶ Repayment status rendering: verify PENDING → amber, PAID → green, OVERDUE → red in Angular data grid
- ▶ Security E2E: attempt to access `/admin/loan-decisions` route as CUSTOMER → verify Angular route guard redirects to `/dashboard`

11. DOCUMENTATION & SPRINT WRAP-UP

Day 30 concludes with a documentation and sign-off session. All deliverables below must be committed to Git before the sprint is considered closed.

11.1 API Documentation

- ▶ Extend springdoc-openapi annotations to cover all new FR-5 /api/loans/... endpoints
- ▶ Annotate LoanProductController, LoanApplicationController, LoanAccountController, LoanRepaymentController with @Operation and @ApiResponse
- ▶ Annotate all new DTOs with @Schema annotations including field descriptions, example values, and constraints
- ▶ Verify Swagger UI at <http://localhost:8080/swagger-ui.html> reflects all 10 new endpoints with correct request/response schemas
- ▶ Export OpenAPI JSON: <http://localhost:8080/v3/api-docs> → commit as docs/openapi-sprint3.json

11.2 Frontend Documentation

- ▶ Document all new Angular components: loans/apply, loans/my-loans, loans/repayment-schedule, admin/loan-products, admin/loan-decisions
- ▶ Document LoanService: methods, HTTP calls, error handling, role-aware conditional rendering
- ▶ Update routing module documentation to include /loans/apply, /loans/my-loans, /admin/loans, /admin/loan-decisions routes
- ▶ Update README.md in neobank-frontend with Sprint 3 features, new Angular Material dependencies, and setup steps

11.3 Sprint Review & Sign-Off

- ▶ Live demonstration of all Sprint 3 deliverables against acceptance criteria in this SRS
- ▶ Code review of both repositories with PMIS mentor — focus on EMI calculation correctness, @Transactional atomicity, and @PreAuthorize security
- ▶ QnA session for Lab 4 and Lab 5 intern teams
- ▶ Formal mentor sign-off upon acceptance of all deliverables
- ▶ Workspace preparation for Milestone 6: Insights, Admin Controls & Stabilization — the final 10-day sprint

12. ACCEPTANCE CRITERIA

All criteria below constitute the Definition of Done for Sprint 3. Every item must be met before the sprint can be signed off.

FR-5 — LOAN PRODUCT CONFIGURATION

#	CRITERION	VERIFIED BY
AC01	Loan product creation restricted to ADMIN role; CUSTOMER access returns HTTP 403 Forbidden	Unit Test
AC02	GET /api/loans/products returns all configured products to any authenticated user	Unit Test + Postman
AC03	Angular Admin product configuration form functional with validation for amount limits and tenure inputs	E2E Test

FR-5 — LOAN APPLICATION & APPROVAL

#	CRITERION	VERIFIED BY
AC04	Loan application API enforces amount within product limits and tenure in allowed list; violations return HTTP 400	Unit Test
AC05	Duplicate PENDING application for same product rejected with HTTP 409 Conflict	Unit Test
AC06	Application ownership resolved exclusively from JWT payload — userId never accepted from request body	Unit Test
AC07	Approval endpoint restricted to ADMIN role; CUSTOMER access returns HTTP 403 Forbidden	Unit Test
AC08	Already-decided application returns HTTP 409 Conflict on repeated decision attempt	Unit Test
AC09	Angular customer loan application form functional with product selection, dynamic validation, and submission	E2E Test
AC10	Angular Admin decision dashboard lists PENDING applications and submits approve/reject decisions correctly	E2E Test

FR-5 — EMI CALCULATION & REPAYMENT

#	CRITERION	VERIFIED BY
AC11	EMI calculated using standard reducing balance formula; result matches expected value for known inputs to 2 decimal places	Unit Test
AC12	LoanAccount creation and full amortization schedule generated atomically within single transaction on loan approval	Unit Test
AC13	Sum of principal components across all instalments equals original loan principal (within rounding tolerance)	Unit Test
AC14	Repayment schedule count matches loan tenure in months exactly	Unit Test + Postman
AC15	Cross-user repayment schedule access rejected with HTTP 403 Forbidden	Unit Test

#	CRITERION	VERIFIED BY
AC16	Angular repayment schedule grid displays correct instalment data with PENDING/PAID/OVERDUE colour coding	E2E Test

GENERAL QUALITY

#	CRITERION	VERIFIED BY
AC17	All unit tests pass with zero critical failures across FR-5	Test Suite
AC18	Swagger UI accessible; all 10 new endpoints documented with request/response schemas	Manual Review
AC19	Git repositories clean: Sprint 3 feature branches merged, commit messages descriptive	Mentor Review
AC20	No hardcoded credentials; EMI logic in dedicated utility class; code follows Infosys formatting standards	Code Review
AC21	README.md files updated in both repos reflecting Sprint 3 features, Angular Material dependencies, and setup changes	Mentor Review

*Prepared under the PMIS (Pradhan Mantri Internship Scheme) · Infosys Bhubaneswar DC · Lab 4 & Lab 5
© Infosys Limited · Confidential · Not for External Distribution*